

1- Challenge Design Document

Objective

First the player is required to log in as an **admin** to a dashboard that already exists on the website and the username is `admin` and the user needs to get the password

The password will be available inside the CTF but it is divided into 3 parts and the user gets each part when they solve one of the 3 vulnerabilities which are **REST**, **GraphQL** and **gRPC**

After collecting all 3 parts and combining them into one password the user can use it to log in to the admin dashboard

But there is one protection point for the flag which is that simply logging into the dashboard by any other method does not mean that the user will be able to see the flag

The user must have completed the 3 conditions related to solving the 3 vulnerabilities in the website and these conditions will be required before the flag can appear and this will be an additional protection layer for the flag

The CTF will be a black box until the user finds the starting point and from there they can start solving and with every progress in solving a vulnerability there will be a small hint for the next vulnerability but it will not directly mention the vulnerability itself

Setup

From here the lab can start

The player runs the CTF and can access it through `localhost:3000` using these two commands

```
npm start
```

```
docker compose up
```

Website Overview

The player will reach the website designed for the challenge and will find a very normal website interface with a login form and also an admin dashboard login that is only available for admins

Here the player will find that they only have the username and are missing the password and there will be a small hint that there is no way to reach this dashboard unless the whole CTF is solved

After that the user will go back to the normal login form and will find that they currently need a username and password to log in and I decided to provide these credentials as a forgotten comment left by the developers in the website and this will be considered the starting point

Vulnerability Flow & Hint System

Here I expect 3 possible scenarios and the user is free to start with any of the 3 vulnerabilities after logging in

The first one is that they perform fuzzing and enumerate all the paths in the website and at that point they will find two paths one called `graph` and the other called `backup` and these two paths are related to the GraphQL and gRPC vulnerabilities

The user can decide which one they want to work on but I will design the whole flow as if it starts from

REST → GraphQL → gRPC

And to make the vulnerabilities more connected and the events more organized there will be small hints after solving each vulnerability but here we cannot predict exactly which vulnerability the user will start with so we will make some conditions for the hints

The 3 vulnerabilities will form a connected circle and the user does not have to start with REST as mentioned before and they can start with any vulnerability

To keep the hints logical there will be a check for which vulnerability the user solved and based on that they will receive a hint for the next vulnerability

For example

- If they start with REST they will get a hint for GraphQL
- If they start with GraphQL they will get a hint for REST

- If they start with gRPC they will get a hint for REST

And the same logic will continue depending on which vulnerability the user solved

Vulnerability 1: REST API — BOLA + Mass Assignment

We will assume that the user did not perform fuzzing and decided to explore the website first and we will start with REST

The vulnerability will be an **IDOR** in the REST API which is **BOLA**

As soon as the user opens the website they will see a profile page containing all of their data including the `id`, `role` and `access_tier` and there will also be another page called `progress` where the password parts will be collected next to each other and the user can copy them and use them on the dashboard login page

The 3 fields mentioned above are the most important 3 things here and one of them is only a decoy which is `role` and the other two are the ones that are supposed to control the user data which are `id` and `access_tier`

To be able to interact with or see the request responsible for this data there is a small function on the website whose only purpose is to change the username and from here the user gets a starting point for modifying and intercepting the request

The first thing the user is expected to think about changing is the `role` and they will find that nothing changes and an error will also appear saying that they are not authorized for something like that

The value of `access_tier` on the website will be `user` and the user will notice the same value in the request intercepted through Burp

From here they modify the value in the request to `admin` and they will see an error saying

Employee ID 76 does not have administrative privileges assigned

From here the user starts to suspect that the ID is wrong and starts brute forcing it using Intruder in Burp Suite from 1 to 100 for example

They will find that ID `62` is the admin and the exploit succeeds and they have now obtained the first password part and solved one vulnerability which is

REST API — BOLA (Broken Object Level Authorization)

Here the user decided to start with the REST vulnerability and in this case they will receive a hint for the GraphQL vulnerability

Sometimes an old service or legacy path remains in the system even after the rest of the services have been updated and secured

Vulnerability 2: GraphQL — Legacy Resolver Authorization Bypass

Here I am hinting that an old service or path is still available and the service is **GraphQL** and version 1 is still accessible

Here the user has no way to solve it except by performing fuzzing as mentioned above and they will discover `graph` and `backup` and we will focus first on `graph`

We will assume that the user will not combine the paths or change direction and will see for example what the `backup` path is about and then start following the `graph` path

Here the vulnerability will be an explicit schema disclosure and from this disclosure the user can expose old fragments or legacy functionality and use them to retrieve data belonging to the admin user and they will also receive a hint for the next vulnerability

After opening `localhost:3000/graph` they will find a very simple page displaying the current versions of the graph service

`v1` and `v2`

If they try to access `v2` they will find that it is highly secured and they will not be able to do anything

Here they will go to `v1` and find a GraphQL IDE page in front of them where they can communicate with GraphQL through queries

Here they can send a normal query directly through the website or through Burp and this is up to the user but in both cases they will be able to reach the solution

There will be a very simple query already sent and it will return a very small part of the schema and also a small hint that they can use the website <https://apis.guru/graphql-voyager/> to work with the GraphQL schema

This website provides a **Copy Introspection Query** button which gives the user a complete query for extracting the schema or they can simply search generally for a ready-made query that displays the full schema of the GraphQL service and this is something very easy to find

After that they will discover old fragments for interacting with the API and learn that they contain 3 fields and they can use them to retrieve the admin data and extract the 3 fields

After displaying all 3 fields they will obtain the GraphQL password part and also receive a hint for the next vulnerability which is the last one and it is **gRPC**

The interfaces you've seen aren't necessarily all the available points of contact; some services listen through separate channels from the usual interfaces

Vulnerability 3: gRPC — JWT Authentication Bypass + Proto File Disclosure

From here the player has a starting point for solving the third and final vulnerability

At this point they know that there is an open port and they are expected to discover it using `nmap` and of course add `-p-` to scan all ports because the gRPC port is `50051` and it appears as unknown

The player is expected to notice it search for it and discover that this port is running a gRPC service and also remember that they found the `backup` path and they are expected to have already checked this path and found a `.proto` file

From here the player starts using the `.proto` definition to understand the required request and send an RPC request to the service running on `50051`

At this stage they will notice that the service requires a **JWT Token** in the request and if they try to send the request without the token it will be rejected and from here they will understand that the JWT is required to access the service

But here the problem is that the service only checks whether the token exists and does not actually verify the validity or contents of the JWT which means that any token can pass the check as long as it is provided in the expected format

Here the player already has a JWT Token that they received when they first logged into the website and therefore they can reuse the same token with the gRPC request instead of needing to obtain a new token

After that they use the `.proto` definition to understand the service method and parameters required and send the request to `50051` using the JWT they already have

This allows them to call the method responsible for internal administrative operations and obtain the final password part

The vulnerability here is mainly a **JWT validation weakness** inside the gRPC service combined with information disclosure through the `.proto` file and the service requires an authentication token but does not properly validate its validity before executing the operation